Snowflake Warehouses — Developer Cheatsheet

Virtual warehouses are Snowflake's compute layer: required for queries, DML, and data load/unload. Per-second billing with a 60-second minimum each time a warehouse starts.

> [!IMPORTANT] Community cheatsheet — not official Snowflake documentation. For the authoritative reference, see Warehouses.

Table of Contents

Sizes & Credits

Gen1 credits per hour (doubles at each size step):

Size	Credits/ hour	Credits/ second	Notes
X-Small	1	0.0003	Default for `CREATE WAREHOUSE` (SQL)
Small	2	0.0006	
Medium	4	0.0011	
Large	8	0.0022	
X-Large	16	0.0044	

Size	Credits/ hour	Credits/ second	Notes
			Default size in Snowsight
2X-Large	32	0.0089	
3X-Large	64	0.0178	
4X-Large	128	0.0356	
5X-Large	256	0.0711	AWS + Azure GA; US Gov preview
6X-Large	512	0.1422	AWS + Azure GA; US Gov preview

> [!NOTE] **Gen2** warehouses are becoming the default (BCR 2026_03, pending). Gen2 uses faster hardware and delivers better analytics and DML performance. Gen1 credit rates above apply; Gen2 rates differ — see the Snowflake Service Consumption Table. X5LARGE and X6LARGE do not support Gen2. **Billing:** 60-second minimum per start. Resize while running adds new compute resources immediately but charges from the resize time.

Warehouse Types

Type	`WAREHOUSE_TYPE`	When to use
Standard (Gen1)	`STANDARD` (default)	General SQL queries, DML, data loading
Standard (Gen2)	`STANDARD` + `GENERATION = '2'`	Analytics and DML workloads; better perf than Gen1
Snowpark-optimized	`SNOWPARK-OPTIMIZED`	ML training, stored procs with large memory needs
Adaptive	`ADAPTIVE`	Analytics/ETL with variable query sizes; no sizing or scaling management needed (Enterprise, AWS preview regions)
Multi-cluster	`STANDARD` + cluster settings	

Type	WAREHOUSE_TYPE	When to use
		High concurrency, many parallel users (Enterprise Edition)

Create: Standard Warehouse

Minimal — defaults to X-Small, Gen1 (or Gen2 if your org/region has BCR 2026_03 enabled):

```
CREATE WAREHOUSE my_wh;
```

Common options in one block:

```
CREATE OR REPLACE WAREHOUSE my_wh
  WAREHOUSE_SIZE       = 'MEDIUM'
  AUTO_SUSPEND         = 300          -- seconds of inactivity
        before suspending
  AUTO_RESUME          = TRUE
  INITIALLY_SUSPENDED = TRUE
  COMMENT              = 'General analytics warehouse';
```

Explicitly create a Gen2 warehouse (recommended for new warehouses):

```
CREATE OR REPLACE WAREHOUSE my_gen2_wh
  WAREHOUSE_SIZE = 'LARGE'
  GENERATION     = '2';
```

Create with query timeout and queue timeout (prevents runaway queries and queue build-up):

```
CREATE OR REPLACE WAREHOUSE my_wh
  WAREHOUSE_SIZE                      = 'MEDIUM'
  AUTO_SUSPEND                        = 300
  STATEMENT_TIMEOUT_IN_SECONDS        = 3600    -- 1 hour max per
        query
  STATEMENT_QUEUED_TIMEOUT_IN_SECONDS = 600     -- 10 min max in
        queue
  INITIALLY_SUSPENDED                 = TRUE;
```

Key `CREATE WAREHOUSE` parameters:

Parameter	Default	What it controls
`WAREHOUSE_SIZE`	X-Small	Compute size; doubles credits at each step
`WAREHOUSE_TYPE`	STANDARD	`STANDARD` or `SNOWPARK-OPTIMIZED`
`GENERATION`		

Parameter	Default	What it controls
	Org/region-dependent	`'1'` or `'2'`; Gen2 = better analytics perf
`AUTO_SUSPEND`	Enabled	Seconds idle before auto-suspend; `0`/`NULL` = never
`AUTO_RESUME`	TRUE	Resume on first query
`INITIALLY_SUSPENDED`	TRUE	Start suspended after CREATE
`MAX_CLUSTER_COUNT`	1	Enterprise: max clusters for scaling out
`MIN_CLUSTER_COUNT`	1	Enterprise: min clusters always running

Create: Snowpark-Optimized Warehouse

Use for Snowpark Python stored procedures, UDFs, and ML training workloads with large memory needs. Takes longer to start than a standard warehouse.

```
CREATE OR REPLACE WAREHOUSE my_ml_wh
  WAREHOUSE_TYPE = 'SNOWPARK-OPTIMIZED'
  WAREHOUSE_SIZE = 'MEDIUM';              -- default
        RESOURCE_CONSTRAINT = MEMORY_16X (256 GB)
```

With explicit memory/CPU configuration:

```
CREATE OR REPLACE WAREHOUSE my_ml_wh
  WAREHOUSE_TYPE      = 'SNOWPARK-OPTIMIZED'
  WAREHOUSE_SIZE      = 'LARGE'
  RESOURCE_CONSTRAINT = 'MEMORY_16X_X86';  -- 256 GB, x86
        architecture
```

`RESOURCE_CONSTRAINT` options for Snowpark-optimized:

Value	Memory	Min size	Availability
`MEMORY_1X` / `MEMORY_1X_X86`	16 GB	X-Small	GA, all regions
`MEMORY_16X` / `MEMORY_16X_X86`	256 GB	Medium	GA, all regions (default)
`MEMORY_64X` / `MEMORY_64X_X86`	1 TB	Large	Preview, AWS only

Create: Adaptive Warehouse

> [!NOTE] **Preview feature — Enterprise Edition required.** Available in AWS regions: US West 2 (Oregon), EU West 1 (Ireland), AP Northeast 1 (Tokyo).

With adaptive warehouses you do **not** set size, multi-cluster counts, QAS, or auto-suspend. Snowflake manages all compute automatically. All adaptive warehouses in an account share a dedicated compute pool and use per-query billing.

Create a new adaptive warehouse (minimal):

```sql
CREATE WAREHOUSE my_adaptive_wh
  WAREHOUSE_TYPE = ADAPTIVE;
```

With performance and throughput tuning:

```sql
CREATE WAREHOUSE my_adaptive_wh
  WAREHOUSE_TYPE               = ADAPTIVE
  MAX_QUERY_PERFORMANCE_LEVEL  = XLARGE
        -- upper bound per query (default XLARGE)
  QUERY_THROUGHPUT_MULTIPLIER  = 2        -- concurrency scale
        factor (default 2, 0 = unlimited)
  COMMENT                      = 'Analytics warehouse — adaptive
        compute';
```

Convert an existing standard warehouse to adaptive (no downtime):

```sql
ALTER WAREHOUSE my_existing_wh SET WAREHOUSE_TYPE = ADAPTIVE;
```

Key parameters:

Parameter	Default	What it controls
`MAX_QUERY_PERFORMANCE_LEVEL`	`XLARGE`	Per-query resource upper bound (XSMALL–X4LARGE)
`QUERY_THROUGHPUT_MULTIPLIER`	`2`	Concurrency scale; `0` = unlimited burst

Enable/disable accepting new jobs (does not drop running queries):

```sql
ALTER WAREHOUSE my_adaptive_wh SUSPEND;   -- stop accepting new
        jobs
ALTER WAREHOUSE my_adaptive_wh RESUME;    -- resume accepting jobs
```

Use & Inspect

```
USE WAREHOUSE my_wh;                    -- set active warehouse for
        session

SHOW WAREHOUSES;                        -- list all warehouses you have
        access to
SHOW WAREHOUSES LIKE 'my_%';          -- filter by name pattern
DESCRIBE WAREHOUSE my_wh;             -- full property list

SHOW PARAMETERS FOR WAREHOUSE my_wh;   -- session and warehouse
        parameters
```

Manage: Resize & Tune

```
ALTER WAREHOUSE my_wh RESUME;         -- manually start a suspended
        warehouse
ALTER WAREHOUSE my_wh SUSPEND;        -- manually stop a running
        warehouse
```

Resize (takes effect for new queries; existing queries keep their resources):

```
ALTER WAREHOUSE my_wh SET
  WAREHOUSE_SIZE       = 'LARGE'
  WAIT_FOR_COMPLETION  = TRUE;        -- wait for existing queries to
        finish before resize
```

Adjust auto-suspend and timeouts:

```
ALTER WAREHOUSE my_wh SET
  AUTO_SUSPEND                        = 120     -- suspend after 2
        min of idle
  STATEMENT_TIMEOUT_IN_SECONDS        = 1800    -- 30 min max per
        query
  STATEMENT_QUEUED_TIMEOUT_IN_SECONDS = 300;    -- 5 min max in
        queue
```

Convert an existing Gen1 warehouse to Gen2:

```
ALTER WAREHOUSE my_wh SET GENERATION = '2';    -- works running or
        suspended
```

Convert standard to Snowpark-optimized:

```
ALTER WAREHOUSE my_wh SET
  WAREHOUSE_TYPE      = 'SNOWPARK-OPTIMIZED'
  RESOURCE_CONSTRAINT = 'MEMORY_1X';
```

> [!NOTE] Converting a running warehouse: existing queries finish on the old type; new queries run on the new type. You are charged for both during the transition period.

Multi-cluster Warehouses

Enterprise Edition feature. Scales out by adding clusters to handle concurrency spikes automatically, without manual resizing.

```
CREATE OR REPLACE WAREHOUSE my_concurrent_wh
  WAREHOUSE_SIZE      = 'MEDIUM'
  MIN_CLUSTER_COUNT   = 1          -- minimum clusters always
        running
  MAX_CLUSTER_COUNT   = 5          -- scale out up to 5 clusters
  SCALING_POLICY      = 'AUTO'     -- 'AUTO' (default) or
        'ECONOMY'
  AUTO_SUSPEND        = 300;
```

Scaling policies:

Policy	Behavior	Use when
`AUTO`	Adds clusters to minimize queuing	Most workloads; prioritizes availability
`ECONOMY`	Fully loads existing clusters before adding new ones	Cost-sensitive; some query queuing is acceptable

Adjust cluster count on a running multi-cluster warehouse:

```
ALTER WAREHOUSE my_concurrent_wh SET
  MIN_CLUSTER_COUNT = 2
  MAX_CLUSTER_COUNT = 8;
```

Resource Monitors

Cap credit spend per warehouse. When the quota is hit, Snowflake can notify, suspend, or suspend immediately.

```
CREATE RESOURCE MONITOR my_daily_cap
  WITH CREDIT_QUOTA  = 50          -- credits per monitoring
        interval
  FREQUENCY          = DAILY
  START_TIMESTAMP    = IMMEDIATELY
  TRIGGERS
    ON 75 PERCENT DO NOTIFY        -- email alert at 75%
    ON 100 PERCENT DO SUSPEND      -- suspend new queries at 100%
    ON 110 PERCENT DO SUSPEND_IMMEDIATE;  -- kill running queries
        at 110%
```

Assign a resource monitor to a warehouse:

```sql
ALTER WAREHOUSE my_wh SET RESOURCE_MONITOR = my_daily_cap;
```

Remove a resource monitor:

```sql
ALTER WAREHOUSE my_wh SET RESOURCE_MONITOR = NULL;
```

Monitor Usage

Check credit consumption for all warehouses over the last 7 days:

```sql
SELECT
  warehouse_name,
  SUM(credits_used)                AS total_credits,
  SUM(credits_used_compute)        AS compute_credits,
  SUM(credits_used_cloud_services) AS cloud_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE start_time >= DATEADD('day', -7, CURRENT_TIMESTAMP())
GROUP BY 1
ORDER BY total_credits DESC;
```

Top warehouses by cost in the last 30 days:

```sql
SELECT
  warehouse_name,
  ROUND(SUM(credits_used), 2) AS credits_30d
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE start_time >= DATEADD('day', -30, CURRENT_TIMESTAMP())
GROUP BY 1
ORDER BY credits_30d DESC
LIMIT 10;
```

Permissions

```sql
GRANT USAGE   ON WAREHOUSE my_wh TO ROLE analyst_role;  -- run queries
GRANT OPERATE ON WAREHOUSE my_wh TO ROLE ops_role;      -- start/stop/resize
GRANT MODIFY  ON WAREHOUSE my_wh TO ROLE admin_role;    -- change parameters
GRANT MONITOR ON WAREHOUSE my_wh TO ROLE dba_role;      -- view load metrics
```

Revoke:

```sql
REVOKE USAGE ON WAREHOUSE my_wh FROM ROLE analyst_role;
```

Drop

```sql
DROP WAREHOUSE my_wh;
```

> [!IMPORTANT] Warehouses cannot be undropped. Verify the name before dropping.

Tips & Gotchas

- **Suspending drops the query cache.** The first queries after a warehouse resumes will be slower while the cache rebuilds. Don't suspend interactive/BI warehouses between queries if sub-second latency matters — set `AUTO_SUSPEND` to match actual gap patterns.
- **60-second minimum per start.** Stopping a warehouse in the first 60 seconds doesn't save credits — you've already been billed for that minute. Only auto-suspend after your minimum gap is consistently longer than 60 seconds.
- **Larger is not always faster.** Small/simple queries don't benefit from X-Large+. Larger warehouses help complex analytical queries, ML training, and high-concurrency loads. Test with your actual workload before sizing up.
- **Gen2 is the new default (pending).** BCR 2026_03 makes Gen2 the default for new warehouses in supported regions. New Gen2 warehouses automatically get Query Acceleration Service (QAS) enabled with `QUERY_ACCELERATION_MAX_SCALE_FACTOR = 2`. Altering a Gen1 warehouse to Gen2 does NOT auto-enable QAS.
- **Multi-cluster requires Enterprise Edition.** `MIN_CLUSTER_COUNT` and `MAX_CLUSTER_COUNT` are silently ignored on lower editions — your warehouse stays single-cluster even if you set `MAX_CLUSTER_COUNT = 5`.

References

- Virtual Warehouses
- Overview of Warehouses
- Gen2 Standard Warehouses
- Snowpark-Optimized Warehouses
- Adaptive Compute
- Multi-cluster Warehouses
- Working with Resource Monitors
- CREATE WAREHOUSE